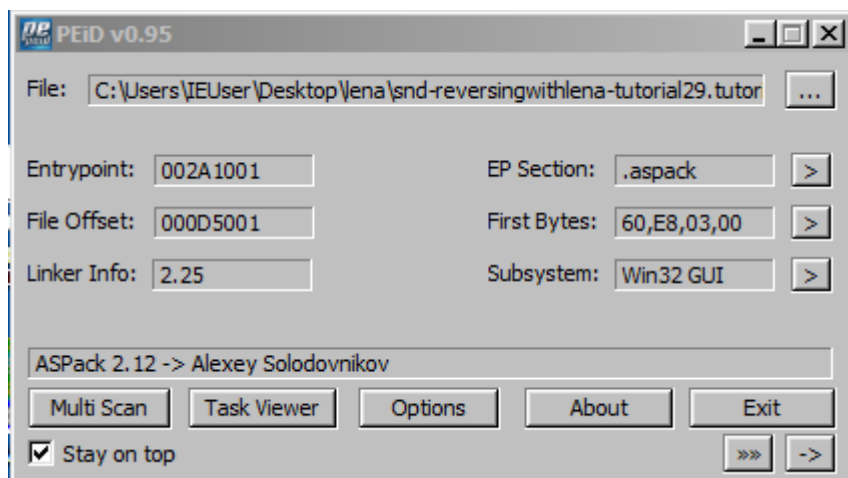


악성코드 분석 보고서

(sand-reversingwithlana-tutorials)

2025.12.29

1. Active MediaMagnet.exe



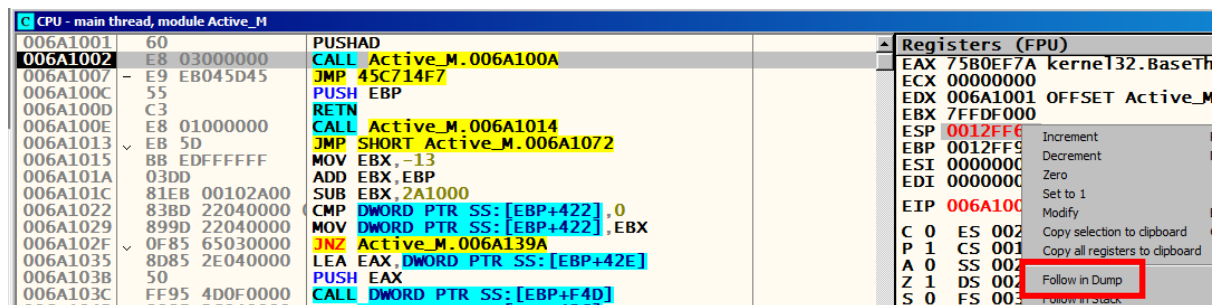
PEiD로 확인 시 ASPack2.12를 사용한 걸 볼 수 있다.

* ASPack 2.12 특징

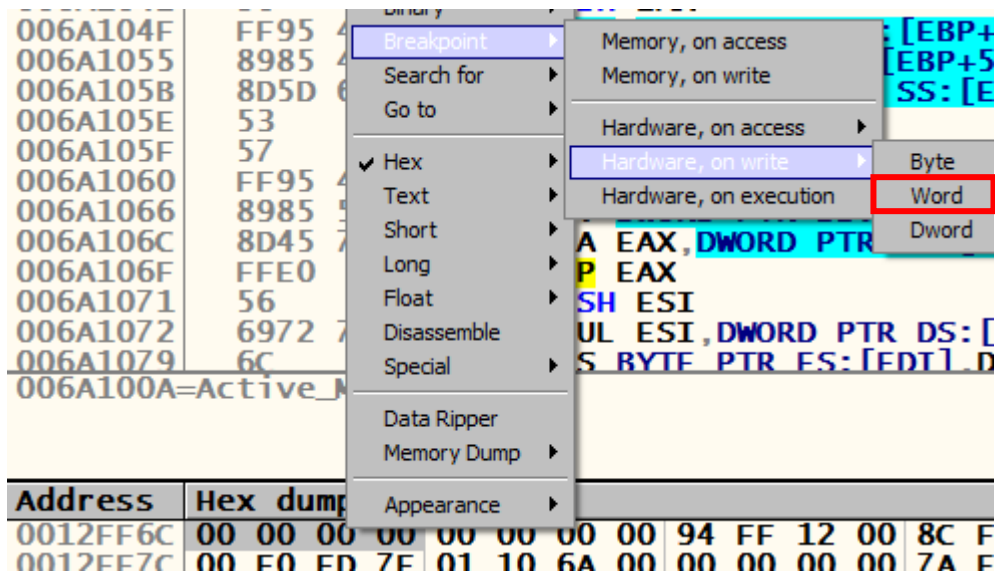
- UPX와 유사한 패킹 기법을 사용함.
- Import Table 거의 다 숨김
- 예외(Exception) 적극 활용
- Anti-Debug은 약함
- 언패킹 루틴 : PUSHAD -> 압축 해제 코드 -> POPAD -> RETN 과정으로 동작

OEP를 찾는 방법이 2가지가 있다. ESP Trick, Sequences 이 있다.

1. ESP Trick 이용



'F8'을 한 번 눌러서 ESP가 변하면 덤프창으로 가준다.



0x0012FF6C에 HwBP를 걸어준다.

006A13B0	75 08	JNZ SHORT Active_M.006A13BA
006A13B2	B8 01000000	MOV EAX,1
006A13B7	C2 0C00	RETN 0C
006A13BA	68 24515E00	PUSH Active_M.005E5124
006A13BF	C3	RETN

'F9'를 누르면 도달하는 걸 볼 수 있다.

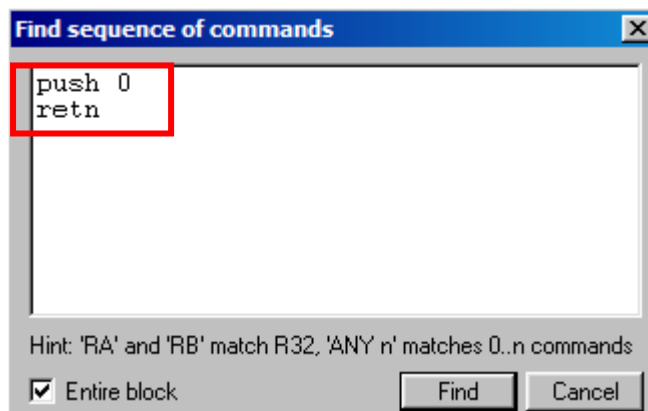
005E5124	. 55	PUSH EBP
005E5125	. 8BEC	MOV EBP,ESP
005E5127	. 83C4 F0	ADD ESP,-10
005E512A	. 33C0	XOR EAX,EAX
005E512C	. 8945 F0	MOV DWORD PTR SS:[EBP-10],EAX
005E512F	. B8 A4485E00	MOV EAX.Active_M.005E48A4

'F8'을 몇 번 눌러주면 OEP를 찾을 수 있다.

2. Sequences로 찾는 방법

Address	Hex	Disassembly
006A1001	60	PUSHAD
006A1002	E8 03000000	CALL
006A1007	E9 EB045D45	JMP
006A100C	55	PUS
006A100D	C3	RET
006A100E	E8 01000000	CALL
006A1013	EB 5D	JMP
006A1015	BB EDFFFFFF	MOV
006A101A	03DD	ADD
006A101C	81EB 00102A00	SUB
006A1022	83BD 22040000	CMPL
006A1029	899D 22040000	MOV
006A102F	0F85 65030000	JNZ
006A1035	8D85 2E040000	LEA
006A103B	50	PUS
006A103C	FF95 4D0F0000	CALL
006A1042	8985 26040000	MOV
006A1048	8BF8	MOV
006A104A	8D5D 5E	LEA
006A104D	53	PUS
006A104E	50	PUS
006A104F	FF95 490F0000	CALL
006A1055	8985 4D050000	MOV
006A105B	8D5D 6B	LEA
006A105E	53	PUS
006A105F	57	PUS
006A1060	FF95 490F0000	CALL
006A1066	8985 51050000	MOV

Search for > All sequences를 눌러준다.



해당 구문은 ASPacker에서 사용되는 전형되는 시퀀스로, 보통 패커 루틴의 끝부분에 위치한다. 즉, OEP를 쉽게 찾을 수 있게 된다.

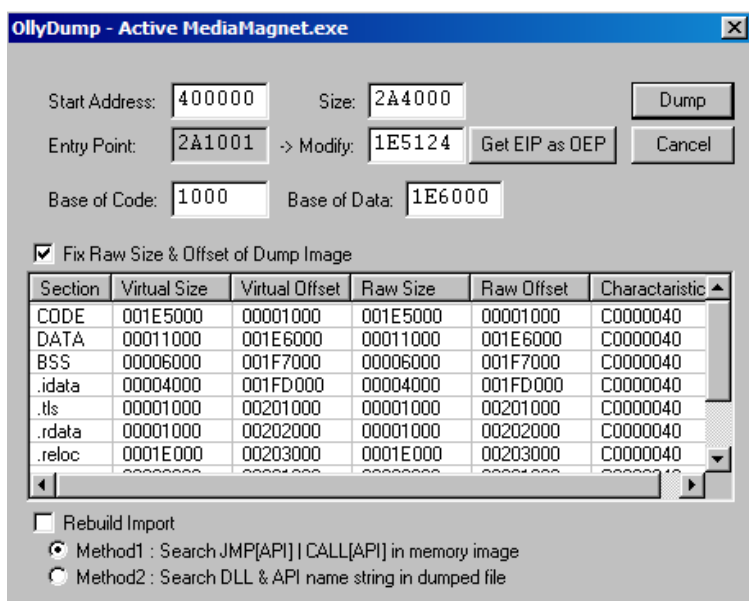
Found sequences		
Address	Disassembly	Comment
006A1001	PUSHAD	(Initial CPU selection)
006A13BA	PUSH 0	

1개가 있는 걸 볼 수 있다. 해당 위치로 넘어가준다.

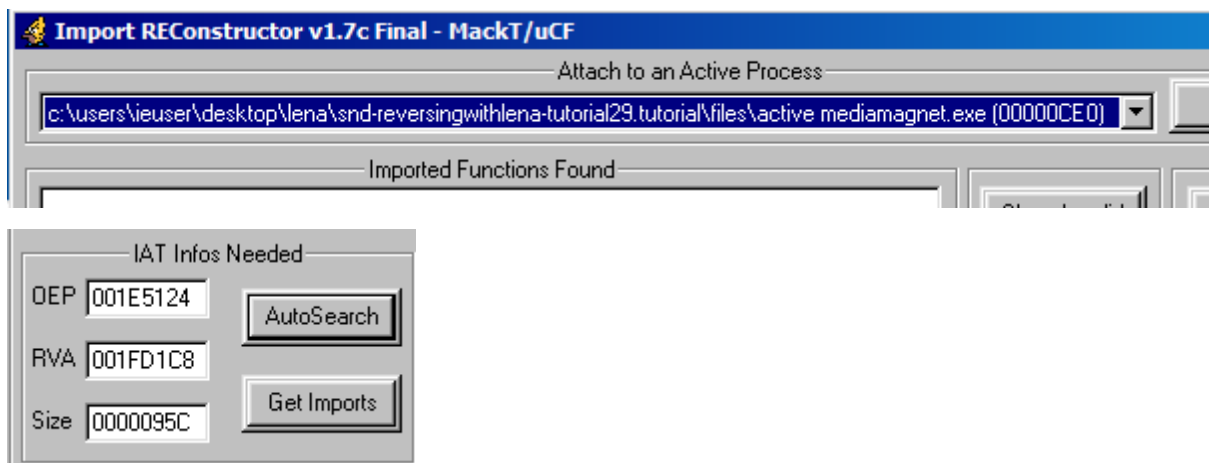
CPU - main thread, module Active_M			
006A13BA	68 00000000	PUSH 0	
006A13BF	C3	RETN	
006A13C0	8B85 26040000	MOV EAX,DWORD PTR SS:[EBP+426]	
006A13C6	8D8D 3B040000	LEA ECX,DWORD PTR SS:[EBP+43B]	

CPU - main thread, module Active_M			
005E5124	. 55	PUSH EBP	
005E5125	. 8BEC	MOV EBP,ESP	
005E5127	. 83C4 F0	ADD ESP,-10	
005E512A	. 33C0	XOR EAX,EAX	

RETN에 BP를 걸어주고 'F9'를 눌러서 넘어가주면 OEP에 도달하는 걸 볼 수 있다.



해당 위치에서 덤프를 떠준다. (Rebuild Import 체크 해제)



ImportREC를 이용하여 IAT를 만들어줄거다. OEP에 값을 넣어주고 AutoSearch를 해주면 RVA와 Size를 얻을 수 있다. 이후 Get Imports를 해준다.

Imported Functions Found	
kernel32.dll	FTThunk:001FD1CC NbFunc:2F (decimal:47) valid:YES
user32.dll	FTThunk:001FD28C NbFunc:4 (decimal:4) valid:YES
advapi32.dll	FTThunk:001FD2A0 NbFunc:3 (decimal:3) valid:YES
oleaut32.dll	FTThunk:001FD2B0 NbFunc:E (decimal:14) valid:YES
kernel32.dll	FTThunk:001FD2EC NbFunc:5 (decimal:5) valid:YES
advapi32.dll	FTThunk:001FD304 NbFunc:C (decimal:12) valid:YES
kernel32.dll	FTThunk:001FD338 NbFunc:68 (decimal:104) valid:YES
version.dll	FTThunk:001FD4DC NbFunc:3 (decimal:3) valid:YES
gdi32.dll	FTThunk:001FD4EC NbFunc:62 (decimal:98) valid:YES
user32.dll	FTThunk:001FD678 NbFunc:C1 (decimal:193) valid:YES
ole32.dll	FTThunk:001FD980 NbFunc:11 (decimal:17) valid:YES
oleaut32.dll	FTThunk:001FD9C8 NbFunc:2 (decimal:2) valid:YES
comctl32.dll	FTThunk:001FD9D4 NbFunc:18 (decimal:24) valid:YES
winspool.drv	FTThunk:001FDA38 NbFunc:5 (decimal:5) valid:YES
shell32.dll	FTThunk:001FDA50 NbFunc:2 (decimal:2) valid:YES
shell32.dll	FTThunk:001FDA5C NbFunc:3 (decimal:3) valid:YES
comdlg32.dll	FTThunk:001FDA6C NbFunc:4 (decimal:4) valid:YES
oledlg.dll	FTThunk:001FDA80 NbFunc:1 (decimal:1) valid:YES
kernel32.dll	FTThunk:001FDA88 NbFunc:1 (decimal:1) valid:YES
kernel32.dll	FTThunk:001FDA90 NbFunc:1 (decimal:1) valid:YES
winmm.dll	FTThunk:001FDA98 NbFunc:1 (decimal:1) valid:YES
wsock32.dll	FTThunk:001FDAA0 NbFunc:20 (decimal:32) valid:YES

전부 YES로 나오니까 바로 덤프를 떠주면 된다.

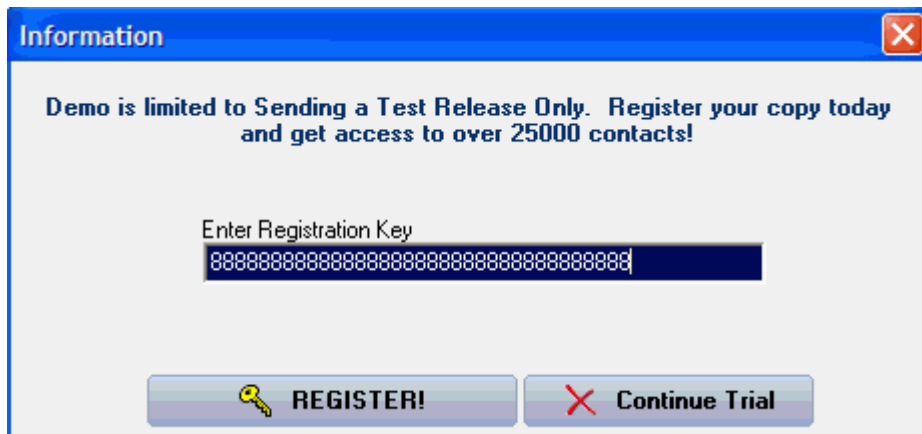
CPU - main thread, module Active_M		
005E5124	\$ 55	PUSH EBP
005E5125	- 8BEC	MOV EBP,ESP
005E5127	- 83C4 F0	ADD ESP,-10
005E512A	- 33C0	XOR EAX,EAX
005E512C	- 8945 F0	MOV DWORD PTR SS:[EBP-10],EAX
005E512F	- B8 A4485E00	MOV EAX,Active_M.005E48A4
005E5134	- E8 D32EE2FF	CALL Active_M.0040800C
005E5139	- 33C0	XOR EAX,EAX
005E513B	- 55	PUSH EBP
005E513C	- 68 87535F00	PUSH Active_M.005E5387

CPU - thread 00000A5C, module KERNELBA		
7584845D	C9	LEAVE
7584845E	C2 1000	RET 10
75848461	8945 C0	MOV DWORD PTR SS:[EBP-40],EAX
75848464	^ EB ED	JMP SHORT KERNELBA.75848453
75848466	3D 01010000	CMP EAX,101
7584846B	^ 0F85 3F93FFFF	JNZ KERNELBA.758417B0
75848471	^ E9 1B93FFFF	JMP KERNELBA.75841791
75848476	8D4D B8	LEA ECX,DWORD PTR SS:[EBP-48]
75848479	FF15 4C108475	CALL DWORD PTR DS:[<&ntdll.RtlDeactivat
7584847F	^ E9 1592FFFF	JMP KERNELBA.75841699

덤프 떠준걸 OllyDbg로 실행시켜준다. 근데 예외처리가 발생하는데 Lena의 영상에서는 예외처리도 안되고 Proxy Setting 전에 레지스터 키 등록키가 나오는데 나는 나오지 않는다.

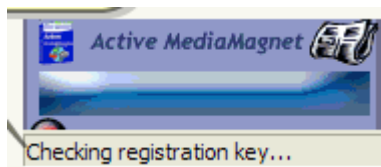
언패킹을 진행했는데도 예외처리가 발생해서 찾아보니 제대로 언패킹이 안되었다. 제대로 된 OEP를 못찾겠어서 Lena 영상으로 대체한다.

-> Lena 영상 참고

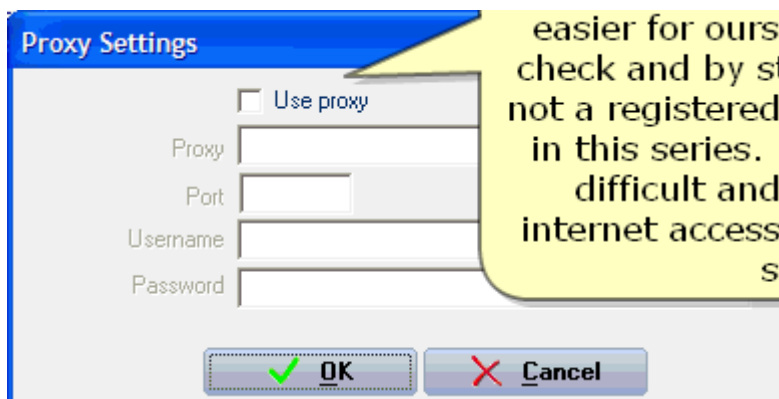


언패킹한걸 Olly로 실행해보면 레지스터 키 입력 칸이 나온다.

체험판이라서 레지스터 키 입력 창이 나오는 걸 볼 수 있다. 등록되었다고 믿게 만들기 위해서 가짜 시리얼 키를 입력해준다.



이렇게 키를 체크하고 있다는 창이 나오는 걸 볼 수 있다.



다음 Proxy Settings 창이 뜨는 걸 볼 수 있다. 왜냐하면 Outbound 트래픽을 차단했기 때문이다. 그래서 해당 웹은 인터넷 접속이 없다고 판단했고, 지금은 우리가 프록시 서버 뒤에 있다고 생각하고 있다. 즉, 인터넷 접속이 차단되면 해당 창이 계속 발생하는 걸 알 수 있다.

서버 체크를 허용해 주면 우리 입장에선 더 쉬울 수도 있겠지만, 그러면 '이건 등록된 시리얼이 아니다(This is not a registered serial)'라는 메시지를 가지고 작업해야 한다.

하지만 이번에는 인터넷 접속을 주지 않고, 실행하려고 한다.

* 소프트웨어가 서버 체크를 진행 할 때 내부적으로 어떤 동작을 수행하는지 예상해 보면 아래와 같다.

1. 내부적으로 소프트웨어가 등록된 버전인지 확인한다. (레지스트리, 기타 다른 방법 이용)

1-1. "등록 된 경우", 서버 체크를 이용하여 시리얼을 검증한다.

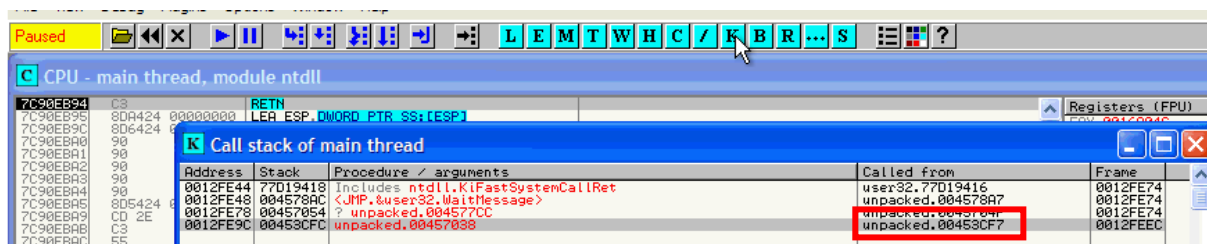
1-2. "등록 안된 경우", 사용자에게 등록을 시킨 다음 서버 체크를 이용한다.

2. 외부 서버와 통신할 수 있는지 체크한다.

2-1. "통신 가능", 서버로 시리얼을 전송하고 응답값을 받아온다.

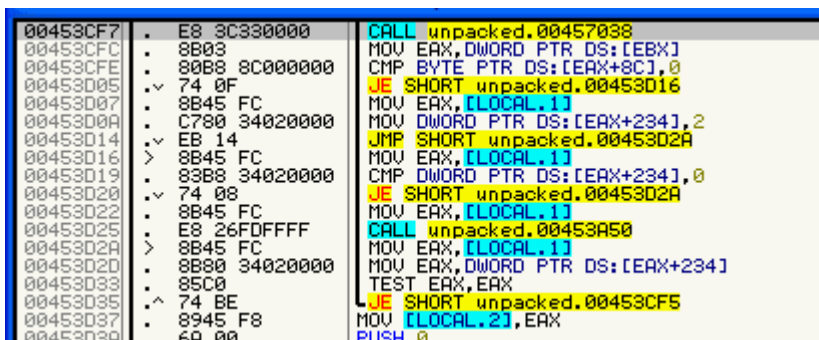
2-2. "통신 불가능", 프록시 설정 창을 보여준다.

3. 다른 경우에 의해 검증이 실패하면 "Trial" 상태로 프로그램을 실행한다

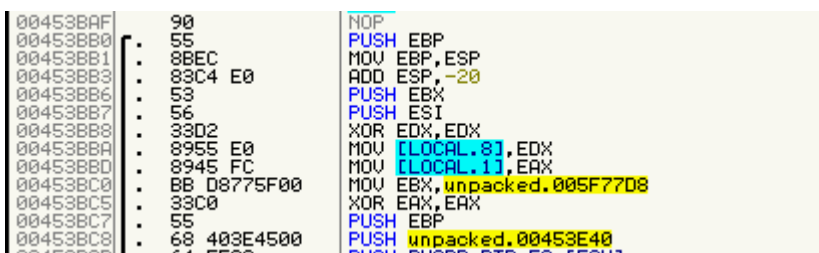


프록시 설정 창이 나타난 상태에서 디버거를 일시정지 시킨 후 콜 스택을 열어본다.

처음에 불러온 함수에 접근한다.



근처 코드를 보면 크게 흥미로운 부분은 없는데, 아마도 프록시 설정 창을 생성하는 코드일 것이다. 스크롤을 위로 올려서 좀 더 이전 부분을 분석해야 한다.



위로 올라갔음에도 흥미로운 부분이 없다. 즉, 이미 Nag 창을 생성하는 코드이기 때문에 좀 더 이전에(일찍) 호출되는 함수를 분석해야 한다. 이 경우 시스템 스택을 이용하여 손쉽게 수행할 수 있다.

00453BAF	90	NOP
00453BB0	55	PUSH EBP
00453BB1	8BE	MOV EBP,ESP
00453BB3	83C4 E0	ADD ESP,-20
00453BB6	53	PUSH EBX

우선 0x00453BB0에 BP를 걸어주고 재실행을 해준다.

00453BB0	55	PUSH EBP
00453BB1	8BEC	MOV EBP,ESP
00453BB3	83C4 E0	ADD ESP,-20

그럼 해당 부분에서 멈추는 걸 볼 수 있다.

K Call stack of main thread			
Address	Stack	Procedure / arguments	Called from

해당 부분에서 콜스택을 보면 아무것도 없는 걸 볼 수 있다.

0012FE44	77D19418	RETURN to user32.77D19418
0012FE48	004578AC	RETURN to unpacked.004578AC from <JMP.&user32.WaitMessage>
0012FE4C	0012FEA0	Pointer to next SEH record
0012FE50	004578C7	SE handler
0012FE54	0012FE74	

이럴 때 스택을 이용한다. 왜냐하면 어플리케이션은 call 명령을 실행할 때, 호출이 끝난 뒤 어디로 돌아가야 하는지 알기 위해 '리턴 주소(return address)'를 스택에 푸시한다.

je = WM_CANCELMODE	0012FF4C	005ABC00	RETURN to unpacked.005ABC00
apture	0012FF50	0012FF5C	Pointer to next SEH record
: 00AFFF4	0012FF54	005ABD55	SE handler

해당 위치를 코드를 보기 위해 Follow in Disassembler를 이용한다.

005ABC75	address	MOV EAX,DWORD PTR SS:[EBP-8]	
005ABC78		MOV EDX,DWORD PTR DS:[EAX]	unpacked.005AA474
005ABC7A	7F52 00000000	CALL DWORD PTR DS:[EDX+08]	unpacked.00453BB0
005ABC80	48	DEC EAX	
005ABC81	0F85 81000000	JNZ unpacked.005ABD08	
005ABC87	8D55 F0	LEA EDX,DWORD PTR SS:[EBP-10]	

DEC EAX위에 Call함수가 있는 걸 볼 수 있다.

[EDX + D8]이라서 Olly가 호출(call) 순서를 재구성하는 데 어려움이 있는 걸 볼 수 있다. 왜냐하면 런타임에 계산되는 간접 호출이기 때문이다.

005ABC88	E8 CFF2FFFF	CALL unpacked.005AA474	
005ABC8D	84C0	TEST AL,AL	
005ABC8F	74 53	JE SHORT unpacked.005ABD04	
005ABC91	6A 00	PUSH 0	
005ABC93	68 A48D5A00	PUSH unpacked.005ABD04	
005ABC98	A1 3805F00	MOV EAX,DWORD PTR DS:[5F6038]	
005ABC9D	FF30	PUSH DWORD PTR DS:[EAX]	
005ABC9F	68 DC8D5A00	PUSH unpacked.005ABD0C	
005ABCC4	8D45 EC	LEA EAX,DWORD PTR SS:[EBP-14]	

ASCII "Thank you! You've successfully registered the
unpacked.005AA474
ASCII " Version!"

해당 코드 밑을 보면 등록 성공했을 때 나오는 문장도 볼 수 있다.

005ABC78	8B10	MOV EDX, DWORD PTR DS:[EAX]	unpacked.005AA474
005ABC7A	FF92 00000000	CALL DWORD PTR DS:[EDX+08]	unpacked.00453BB0
005ABC80	48	DEC EAX	

005ABBE3	55	PUSH EBP
005ABBE4	8BEC	MOV EBP, ESP
005ABBE5	33C9	XOR ECX, ECX
005ABBE9	51	PUSH ECX
005ABBEA	51	PUSH ECX
005ABBEB	51	PUSH ECX
005ABBEF	51	PUSH ECX
005ABBEF	51	PUSH ECX
005ABBEF	53	PUSH EBX
005ABBEF	56	PUSH ESI
005ABBF0	8BF0	MOV ESI, EAX
005ABBF2	33C0	XOR EAX, EAX
005ABBF4	55	PUSH EBP

우선 call함수에 BP를 걸어준 후 위로 올라가서 시작 부분으로 간다.

005ABBF0	8BF0	MOV ESI, EAX
005ABBF2	33C0	XOR EAX, EAX
005ABBF4	55	PUSH EBP
005ABBF5	68 8CB05A00	PUSH unpacked.005ABD8C
005ABBF6	64:FF30	PUSH DWORD PTR FS:[EAX]
005ABBF7	64:8920	MOV DWORD PTR FS:[EAX], ESP
005ABC00	C645 FF 00	MOV BYTE PTR SS:[EBP-1], 0
005ABC04	8BC6	MOV EAX, ESI
005ABC06	E8 81F2FFFF	CALL unpacked.005AAE8C
005ABC08	8BD8	MOV EBX, EAX
005ABC0D	889E 11030000	MOV BYTE PTR DS:[ESI+311], BL
005ABC13	84DB	TEST BL, BL
005ABC15	0F84 41010000	JE unpacked.005ABD5C
005ABC1B	8D55 F4	LEA EDX, DWORD PTR SS:[EBP-C]
005ABC1E	8BC6	MOV EAX, ESI
005ABC20	E8 FFFDFFFF	CALL unpacked.005ABA24
005ABC25	33D2	XOR EDX, EDX
005ABC27	55	PUSH EBP
005ABC28	68 55B05A00	PUSH unpacked.005ABD55

JE 조건문이 있는 걸 볼 수 있는데 위에 BL을 이용해서 비교하는 걸 볼 수 있다. 해당 분기문이 도달하는 구역까지 내려가보면 등록 성공했을 때 나오는 문장을 넘어가는 걸 볼 수 있다.

우선 0x005ABC0D에 BP를 걸어주고 재시작을 해준다.

005ABC0D	889E 11030000	MOV BYTE PTR DS:[ESI+311], BL
005ABC13	84DB	TEST BL, BL
005ABC15	0F84 41010000	JE unpacked.005ABD5C
005ABC1B	8D55 F4	LEA EDX, DWORD PTR SS:[EBP-C]
005ABC1E	8BC6	MOV EAX, ESI
005ABC20	E8 FFFDFFFF	CALL unpacked.005ABA24
005ABC25	33D2	XOR EDX, EDX
005ABC27	55	PUSH EBP
005ABC28	68 55B05A00	PUSH unpacked.005ABD55
005ABC2D	64:FF32	MOV DWORD PTR SS:[EBP-2], 0
005ABC30	64:8922	MOV DWORD PTR FS:[EAX], ESP
005ABC33	8BCE	MOV ECX, EDI
005ABC35	B2 01	MOV EBX, 1

005ABD02	EB 31	JMP SHORT unpacked.005ABD35	jumps
005ABD04	B0 01	MOV AL, 1	
005ABD06	EB 2D	JMP SHORT unpacked.005ABD35	
005ABD08	BA F4B05A00	MOV EDX, unpacked.005ABD0F4	
005ABD0D	8BC6	MOV EAX, ESI	
005ABD0F	E8 38FCFFFF	CALL unpacked.005AB894C	
005ABD14	BA F4B05A00	MOV EDX, unpacked.005ABD0F4	ASCII "trial"
005ABD19	8BC6	MOV EAX, ESI	ASCII "trial"
005ABD1B	E8 54FBFFFF	CALL unpacked.005AB874	
005ABD20	C645 FF 01	MOV BYTE PTR SS:[EBP-1], 1	
005ABD24	A1 38605F00	MOV EAX, DWORD PTR DS:[5F6038]	
005ABD29	BA F4B05A00	MOV EDX, unpacked.005ABD0F4	
005ABD2E	E8 6982E5FF	CALL unpacked.00403F9C	ASCII "trial"
005ABD33	33C0	XOR EAX, EAX	

재시작 후 JE까지 도달하면 분기가 일어나지 않는 걸 보인다. 쪽 내려가면 등록 성공했을 때 나오는 문장에 도달하는 걸 볼 수 있고, 또한 체험판 문자열 부분도 넘어가는 걸 볼 수 있다.

```

005ABD5A . 8B EE JMP SHORT unpacked.005ABD4A
005ABD5C . 8BC6 MOV EAX,ESI
005ABD5E . E8 19F2FFFF CALL unpacked.005AAF7C
005ABD63 . 8845 FF MOV BYTE PTR SS:[EBP-1],AL
005ABD66 . 33C0 XOR EAX,EAX
005ABD68 . 5A POP EDX
005ABD69 . 59 POP ECX
005ABD6A . 59 POP ECX
005ABD6B . 64:8910 MOV DWORD PTR FS:[EAX],EDX
005ABD6E . 68 93BD5A00 PUSH unpacked.005ABD93
005ABD73 . 8D45 EC LEA EAX,DWORD PTR SS:[EBP-14]
005ABD76 . E8 CD81E5FF CALL unpacked.00403F48
005ABD7B . 8D45 F0 LEA EAX,DWORD PTR SS:[EBP-10]
005ABD7E . E8 C581E5FF CALL unpacked.00403F48
005ABD83 . 8D45 F4 LEA EAX,DWORD PTR SS:[EBP-C]
005ABD86 . E8 BD81E5FF CALL unpacked.00403F48
005ABD8B . C3 RETN
005ABD8C . E9 AF7BE5FF JMP unpacked.00403940
005ABD91 . EB E0 JMP SHORT unpacked.005ABD73
005ABD93 . 8A45 FF MOV AL,BYTE PTR SS:[EBP-1]
005ABD96 . 5E POP ESI
005ABD97 . 5B POP EBX
005ABD98 . 8BE5 MOV ESP,EBP
005ABD9A . 5D POP EBP
005ABD9B . C3 RETN

```

And here
of the

아무런 조건문 없이 함수 종료를의미하는 RETN 까지 실행되는 걸 볼 수 있다.

```

005ABC81 . 0F85 81000000 JNZ unpacked.005ABD08
005ABC87 . 8D55 F0 LEA EDX,DWORD PTR SS:[EBP-10]
005ABC8A . 8B45 F8 MOV EAX,DWORD PTR SS:[EBP-8]
005ABC8D . 8B80 E4020000 MOV EAX,DWORD PTR DS:[EAX+2E4]
005ABC93 . E8 10C7E8FF CALL unpacked.004383A8
005ABC98 . 8B55 F0 MOV EDX,DWORD PTR SS:[EBP-10]
005ABC9B . 8D86 00030000 LEA EAX,DWORD PTR DS:[ESI+308]
005ABCA1 . E8 F682E5FF CALL unpacked.00403F9C
005ABCA6 . 8BC6 MOV EAX,ESI
005ABCA8 . E8 CFF2FFFF CALL unpacked.005AAF7C
005ABCAD . 84C0 TEST AL,AL
005ABCAD . 74 53 JE SHORT unpacked.005ABD04
005ABCAD . 6A 00 PUSH 0
005ABCAD . 68 A4BD5A00 PUSH unpacked.005ABD44
005ABCAD . A1 38605F00 MOV EAX,DWORD PTR DS:[5F6038]
005ABCAD . FF30 PUSH DWORD PTR DS:[EAX]
005ABCAD . 68 DCBD5A00 PUSH unpacked.005ABD0C
005ABCAD . 8D45 EC LEA EAX,DWORD PTR SS:[EBP-14]
005ABCAD . BA 03000000 MOV EDI,3

```

ASCII "Thank you! You've successfully registered the "

ASCII " Version!"

해당 부분은 사용자가 등록하려고 할 때 실행되는 루틴이다.

```

005ABCF7 . E8 0CFEFFFF CALL unpacked.005ABB08
005ABCF8 . C645 FF 01 MOV BYTE PTR SS:[EBP-1],1
005ABD00 . 33C0 XOR EAX,EAX
005ABD02 . EB 31 JMP SHORT unpacked.005ABD35
005ABD04 . B0 01 MOV AL,1
005ABD06 . EB 20 JMP SHORT unpacked.005ABD35
005ABD08 . BA F4BD5A00 MOV EDI,unpacked.005ABD44
005ABD0D . 8BC6 MOV EAX,ESI
005ABD0F . E8 38FCFFFF CALL unpacked.005AB94C
005ABD14 . BA F4BD5A00 MOV EDI,unpacked.005ABD44
005ABD19 . 8BC6 MOV EAX,ESI
005ABD1B . E8 54FBFFFF CALL unpacked.005AB874
005ABD20 . C645 FF 01 MOV BYTE PTR SS:[EBP-1],1
005ABD24 . A1 38605F00 MOV EAX,DWORD PTR DS:[5F6038]

```

ASCII "trial"

ASCII "trial"

등록이 실패하면 체험판(trial) 상태로 계속 가는 부분인 걸 볼 수 있다.

```

005ABC00 . 8B08 MOV EBX,EAX
005ABC00 . 889E 11030000 MOV BYTE PTR DS:[ESI+311],BL
005ABC13 . 84DB TEST BL,BL
005ABC15 . 0F84 41010000 JE unpacked.005ABD5C
005ABC18 . 8D55 F4 LEA EDX,DWORD PTR SS:[EBP-C]
005ABC1E . 8BC6 MOV EAX,ESI
005ABC20 . E8 FFDFFFFF CALL unpacked.005ABA24

```

여기까지 생각해보면 0x005ABC15 분기문은 이미 등록됐는지 확인하는 코드로 볼 수 있다. (만약 사용자가 등록되어 있다면 전체 등록 루틴과 시리얼 검증을 전부 건너뛰는 걸 알 수 있다.)

* 우리가 프로그램에게 '처음부터 등록된 상태다'라고 말해주더라도, 서버 체크는 여전히 실행된다.

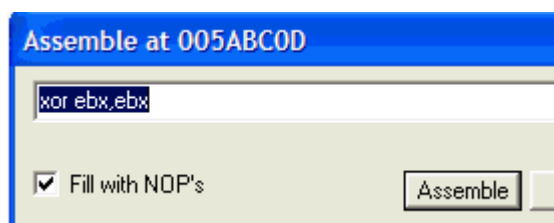
005ABC06	. E8 81F2FFFF	CALL unpacked.005AAE8C
005ABC08	8B08	MOV EBX,EBX
005ABC0D	. 889E 11030000	MOV BYTE PTR DS:[ESI+311],BL
005ABC13	. 84DB	TEST BL,BL
005ABC15	.v 0F84 41010000	JE unpacked.005ABD5C
005ABC1B	. 8D55 F4	LEA EDX,DWORD PTR SS:[EBP-C]
005ABC1F	. 8BC6	MOV FAX,FST

아마 해당 함수가 이미 등록된 상태인지 검증하는 함수일 것이다. 하지만 이 부분은 중요하지 않다. 왜냐하면 이 프로그램은 어차피 등록 여부와 상관없이 서버 체크를 실행하기 때문이다.

해당 call함수를 패치해서 그 호출 안에서 BL(사실 이 호출에서는 EAX다)이 어디서 설정되는지, 왜 그렇게 되는지를 찾아볼 수도 있다.

하지만 이번에는 '단순 무식한 패칭 방법'을 사용할거다.

TEST BL, BL -> XOR BL, BL로 변경해주는 방법 or JE를 항상 점프하도록 패치해주는 방법이다. 해당 방법은 MOV EBX, EAX -> XOR EBX, EBX로 바꿔주면 된다.



005ABC06	. E8 81F2FFFF	CALL Unpacked.005AAE8C
005ABC08	33DB	XOR EBX,EBX
005ABC0D	. 889E 11030000	MOV BYTE PTR DS:[ESI+311],BL
005ABC13	. 84DB	TEST BL,BL
005ABC15	.v 0F84 41010000	JE Unpacked.005ABD5C
005ABC1B	. 8D55 F4	LEA EDX,DWORD PTR SS:[EBP-C]

위 사진의 주소와 명령어를 나중에 인라인 패치로 진행하기 위해 기억해 두어야 한다.

-> 왜 패치가 안되는지 이해가 다 안됨,,, 나중에 다시,, 레나는 언패킹을 했는데,,?